

phorce 매뉴얼

참가자가 쓰는 명령·API·인터페이스의 체계적 참조. 개념 설명은 퀵 가이드, 따라하기는 튜토리얼을 보세요. 이 문서는 "정확히 뭐가 있고 어떻게 부르나"의 사전입니다.

차례 1. 시스템 개요 · 2. CLI · 3. GUI · 4. Python API · 5. C++ API · 6. ROS 2 인터페이스 · 7. 피드백 필드 · 8. 모션

슬롯 계약 · 9. 거절 코드 · 10. 안전 · 11. 흔한 실수 · 12. 참가자 API 가 아닌 것 · 13. 용어집

1. 시스템 개요

Jetson(젯슨, 여러분의 컴퓨터) — **pcm**(중계기) — **phact**(관절 모터 — 와이어 12 축 중 6 축 장착). 참가자가 만지는 표면은 **두 방향**뿐입니다.

- **↑ 피드백**: 로봇이 12 축 상태를 **1kHz**(1 초 1000 번)로 올려보냄 (`/phorce/feedback`)
- **↓ 모션 재생**: 참가자는 **모션 ID(1~50)** 하나를 보냄 → 미리 저장된 동작을 로봇이 재생

모션 궤적은 행사 전 **phorce Studio** 로 만들어 로봇(pcm)에 저장됩니다. 참가자 코드는 관절을 직접 제어하지 않고 **재생할 동작 번호만** 고릅니다.

2. CLI 레퍼런스 — phorce

파이썬으로 만든 명령줄 도구. 서브커맨드는 **4 개**입니다.

명령	하는 일	기본 타임아웃
<code>phorce doctor</code>	게이트웨이·카탈로그·상태 진단 한 장	2 초
<code>phorce list</code>	재생 가능한 모션 목록 (정본 = 로봇 적재 슬롯)	5 초

`phorce play <id>` 모션 하나 재생, 완료까지 대기 (진행률 표시) 30 초

`phorce status` 모션 슬롯 상태 한 장 (읽기 전용) 2 초

공통 플래그 (모든 명령): `--target`(기본 `robot` = 실물 — 그대로 두면 됨), `--namespace`, `--domain-id`, `--timeout <초>`, `--json`(기계 판독용).

종료 코드 (스크립트에서 유용):

코드	뜻	코드	뜻
0	성공	3	게이트웨이 없음
1	거부/실패/준비안됨	4	타임아웃
2	사용법 오류	5	BUSY (요청 폐기, 큐 없음)
		130	사용자 취소(Ctrl+C)

`phorce list` 의 목록은 젯슨 파일이 아니라 **로봇이 실제 적재한 슬롯**(0x4202 비트맵)입니다. 목록에 뜨면 재생 가능, 안 뜨면 불가.

3. GUI — `phorce-console`

관측·전송용 샘플 화면 (PyQt5). 실행: `phorce-console`. 별도 API 가 아니라 참가자와 **똑같은 모션 액션**을 부르는 ROS 2 노드입니다.

- 12 축 위치/속도/전류/온도 관측, 시계열 그래프(동시 8 채널), 신선도·E-Stop·EtherCAT 상태
- 모션 슬롯 버튼 전송 — **대상 칸을 ● 실물 로봇으로 바꿔 사용** (기본값 "시뮬레이터"는 이 행사에서 쓰지 않음).
실물 선택 시 테두리 빨강 + 버튼 **[실물 전송]** + 경고문 (3 중 동시)
- 자동 반복 재생 기능 없음. `python3-pyqtgraph` 없으면 그래프만 비활성(관측·전송은 동작)

4. Python API

여기는 요약입니다 — 전체 시그니처·클래스별 메서드·필드 사전은 [04-api-reference의 Python 탭](#)을 보세요.

4-1. phorce 파사드 (권장, 가장 쉬움)

```
import phorce
with phorce.connect() as robot:      # 기본 target=robot(실물) — 그대로 쓰면 됨
    result = robot.play(1)           # 완료까지 블로킹
    print(result.ok)                 # ★ 속성(attribute)입니다. ok() 아님
    for m in robot.motions():        # 카탈로그 순회
        print(m.id, m.name)
    handle = robot.play_async(2, on_feedback=cb) # 비동기
    handle.wait(timeout=30)
```

공개 심볼: `connect`, `doctor`, `Robot`, `Target`, `Motions`, `Motion`, `Catalog`, `Feedback`, `Status`, `PlayHandle`, `PlayResult`.

예외: `PhorceError`(최상위), `PhorceUnavailable`, `MotionBusy`, `MotionRejected`, `MotionAborted`.

상수: `MIN_MOTION_ID`(1), `MAX_MOTION_ID`(50), `NO_MOTION_ID`(0).

거절/실패는 조용히 지나가지 않고 예외로 옵니다. `try/except` 로 `MotionBusy`(기다림) / `MotionRejected`(사람 개입)만 구분하면 충분합니다.

4-2. 표준 rclpy (직접 구독)

```
from rclpy.qos import qos_profile_sensor_data # ★ 필수
from agx_msgs.msg import PhorceFeedback
node.create_subscription(PhorceFeedback, "/phorce/feedback", cb, qos_profile_sensor_data)
```

QoS 함정: `/phorce/feedback` 는 반드시 `qos_profile_sensor_data`(best-effort)로 구독. 기본 `reliable` 로 하면 에러 없이 한 개도 안 옵니다.

5. C++ API — `phorce_cpp::motion_client`

여기는 요약입니다 — 전체 시그니처와 구조체 필드는 [04-api-reference의 C++ 탭](#)을 보세요.

```
find_package(phorce_cpp REQUIRED)
target_link_libraries(x phorce_cpp::motion_client) // 공개 의존: rclcpp, rclcpp_action, agx_msgs
```

타입 / 메서드	설명
<code>MotionClient::attach(node, Target)</code>	클라이언트 생성. 스스로 spin 안 함(executor 는 호출자 몫)
<code>play_async(id, cb={})</code>	동작 요청 → <code>shared_ptr<PlayOperation></code>
<code>motions_async()</code>	카탈로그 조회 → future
<code>latest_status(), action_ready(), wait_for_action_server()</code>	상태·준비 확인
<code>PlayOperation::result()</code>	<code>shared_future<PlayResult></code>
<code>PlayOperation::cancel()</code>	취소 (E-Stop 아님 — 실물에선 거부될 수 있음)
<code>PlayResult::ok()</code>	성공 여부
<code>PlayResult::busy()</code>	큐 가득참(코드 5)일 때만 참 → 재시도 루프 조건
<code>PlayResult::needs_operator()</code>	코드 12·13 (사람이 버튼)
<code>Target::robot()</code>	대상 선택 (기본값 — 실물)

상수: `kMinMotionId=1`, `kMaxMotionId=50`, `kNoMotionId=0`, `kMaxSequenceLength=1`, `kStateFreshLimitMs=1500`.

콜백 안에서 `future.get()` 으로 블로킹하지 마세요(자기 자신을 굶킵니다). 타이머/루프에서 `wait_for(0ms)` 로 논블로킹 확인하세요.

6. ROS 2 인터페이스

이름	종류 / 타입	속도·QoS	용도
----	---------	--------	----

<code>/phorce/feedback</code>	topic · <code>PhorceFeedback</code>	1kHz · sensor_data	↑ 12 축 실시간 상태 (참가자 입력)
<code>/phorce/status</code>	topic · <code>PhorceStatus</code>	10Hz · reliable	모드 · 지터 · 카운터 요약
<code>/phorce/motion_window</code>	topic · <code>MotionWindowState</code>	2Hz · latched	적재 슬롯 비트맵 · busy 등
<code>.../play_motion_sequence</code>	action · <code>PlayMotionSequence</code>	—	↓ 참가자 유일 구동 API
<code>.../motion_slot_state</code>	topic · <code>MotionSlotState</code>	—	모션 슬롯 상태 (읽기 전용, 폴링→발사 금지)
<code>~/list_motion_slots</code>	service · <code>ListMotionSlots</code>	—	카탈로그 조회

Raw 액션 호출 (파사드 없이, CLI 와 동일 경로):

```
ros2 action send_goal /motion_action_server/play_motion_sequence \
  agx_msgs/action/PlayMotionSequence "{motion_ids: [1], stop_on_error: true}" -f
```

7. 피드백 필드 (`PhorceFeedback` / `AxisFeedback[12]`)

프레임당: `stamp`, `wkc`, `tx_cycle_seq`, `axis_valid_mask`, `axis_stale_mask`, `axis_oper_mask`, `axis_fault_mask`, `am_rx_age_ms`, `status_flags`, `axis[12]`.

축당 필드	뜻
<code>position_rad</code> / <code>velocity_rad_s</code>	관절 각도(rad) / 각속도(rad/s)
<code>current_a</code> / <code>dob_a</code>	모터 전류(A) / 외란 추정(A)
<code>bus_v</code> / <code>temp_c</code>	전압(V) / 온도(°C)
<code>kp_echo</code> / <code>kd_echo</code>	적용 중 게인 되움림 (<code>kp</code> =A/rad, <code>kd</code> =A/(rad/s) — 전류 도메인)
<code>valid</code>	이 축을 믿어도 되는 유일한 양(+)의 증거. <code>!stale</code> 로 대체 금지

8. 모션 슬롯 계약

- 재생 가능 ID: **1..50**
- **0** = no-motion sentinel — **play(0)** 금지
- **motion_00.csv** = 템플릿, 카탈로그·재생에서 제외
- 요청 하나당 ID 하나 (**max_sequence_length = 1**)
- 카탈로그 정보 = **로봇(pcm)** 이 적재한 슬롯. 젯슨 파일 아님

9. 거절 코드 (PlayMotionSequence reject_reason)

코드	이름	대응
5	QUEUE_FULL / BUSY	기다리면 풀림. 재시도 루프는 이 코드에서만
12	NOT_READY_FOR_MOTION	사람: 영점 버튼(1 번) 0.6 초 → 3 초 대기
13	RECOVERY_REQUIRED	사람: 2 번 버튼 파킹 → 다시 영점
4	MOTION_ID_NOT_LOADED	phorce list 에 있는 id 사용
3	MOTION_ID_RANGE	1~50 범위로
6·11	MASTER_NOT_OP·AXIS_NOT_OPERATIONAL	EtherCAT/축 상태 확인 (배선·전원)

0·1·2·7·8·	NONE/EMPTY/TOO_LONG/COMMAND_SOURCE/STATE_STALE/	요청 형식·상태 문제 — 메시지
9·10	SUPERVISOR_VETO/CONTRACT_NOT_ACTIVE	detail 참조

재시도 루프는 코드 5(BUSY)에서만 돌리세요. 12·13 은 기다려도 안 풀립니다 — 사람이 버튼을 눌러야 합니다.

표의 이름은 줄인 표기입니다 — 정식 상수와 Python `.reason` 문자열에는 `REJECT_` 접두사가 붙습니다 (예: `REJECT_QUEUE_FULL`). 코드에서 비교할 때는 숫자(`.code`)가 안전합니다.

10. 안전

- 비상 정지는 물리 **E-Stop** 버튼 하나뿐. 코드의 `cancel()` 은 E-Stop 이 아님
- 실물 전송 전 항상 로봇 주변 확인
- 모든 하행 명령은 게이트웨이의 **안전 감시자** 6 단계(하드 veto→신선도→NaN→한계→슬루)를 통과해야 나감
- 참가자 API(모션 슬롯)는 이 감시자를 우회할 수 없음 — 안전하게 설계됨

11. 흔한 실수

증상	원인·해결
doctor 가 NOT READY / "서버 없음"	스택(터미널 2 개)을 안 켜거나 닫음 — ① 퀵 가이드 3-2 절 시동 걸기
모션 목록·상태가 계속 빈 채	로봇 전원보다 스택을 먼저 켜음 — 터미널 1(monitor) 재시작
버튼 복구 직후 첫 재생이 "중단됨 (error=17)"	상태 갱신(0.5 초 주기)이 따라오기 전 — 한 번 더 보내면 됩니다
피드백이 하나도 안 옴(에러도 없음)	QoS 누락 → <code>qos_profile_sensor_data</code> 로 구독
거의 다 BUSY 로 거절	빠른 콜백에서 <code>play()</code> 호출 → 느린 루프로 옮기기 (큐 없음)
계속 "준비 안 됨"	영점 버튼(1 번) 안 눌림 — 사람이 눌러야
<code>phorce: command not found</code>	재로그인. 그래도 없으면 그 보드 재검수

C++ 재빌드 후 권한 오류

`sudo agr-setcap-ethercat <실행파일>` 재실행(저수준만 해당)

로봇이 안 움직임(에러 없음)

GUI 대상 칸이 기본값 "시뮬레이터"로 남아 있는 경우가 대부분 — ● **실물**
로봇으로 변경

12. 참가자 API 가 아닌 것 (의도적 비공개)

다음은 참가자용이 아닙니다. 이것 없이도 로봇을 충분히 다룰 수 있게 설계됐습니다.

- 관절 1kHz 직접 스트리밍, raw PDO/SDO 접근
- 레시피 4 량({목표위치, 피드포워드토크, Kp, Kd}) 저수준 명령 — 내부/벤치 전용
- CSV·P/F/I Vector 편집 — 이것은 행사 전 phorce Studio 의 몫

13. 용어집

Jetson (젯슨) 여러분이 코드를 돌리는 컴퓨터(뇌)

pcm

Jetson 과 모터 사이 중계기(신경). EtherCAT 슬레이브 — 모션 슬롯을 적재·재생하는 주체(카탈로그
정본)

phact

관절마다 붙은 모터 장치(근육). 와이어 12 축 중 현 기체는 6 축 장착

모션 슬롯 / ms_id 미리 저장된 동작 하나와 그 번호(1~50)

피드백

`/phorce/feedback` — 12 축 상태를 1kHz 로 올리는 스트림

게이트웨이

참가자 요청을 받아 안전 검사 후 로봇에 전달하는 보호 계층

EtherCAT

Jetson-pcm 을 잇는 산업용 실시간 통신(1kHz)

phorce Studio

행사 전 모션 궤적을 만들어 로봇에 저장하는 도구(참가자 미사용)

영점 버튼

기체의 1 번 버튼. 로봇을 모션 수신 상태로 만드는 물리 버튼

phorce 해커톤 참가자 문서 ③ 매뉴얼 — 함께 보기: ① 퀵 가이드 · ② 튜토리얼 · ④ API 레퍼런스(함수·필드 사전) · ⑤ 시스템 안내